

Week5_Yuliia_Chernysheva

February 14, 2026

Yuliia Chernysheva

Date: February 14, 2026

Instructions

Part1: Functions

* Write a program to prompt the user for hours and rate per hour using input to compute gross pay. Award time-and-a-half for the hourly rate for all hours worked above 40 hours. Put the logic to do the computation of time-and-a-half in a function called **compute_pay** and use the function to do the computation. **The function should return a value.**

* The function should return a value. Use **45 hours** and a rate of **10.50 per hour** to test the program (**the pay should be 498.75**). You should use input to read a string and float() to convert the string to a number.

* Do not worry about error checking the user input unless you want to - you can assume the user types numbers properly. Do not name your variable sum or use the sum() function.

Part2: Lists

* Open the file **romeo.txt** and read it **line by line**. For each line, split the line into a Python List using the **split()** method. The program should build a **list of words**. When the program completes, **sort** (default sort) and **print** (horizontal or vertical print) the resulting words in alphabetical order.

Part3: Dictionaries

* Write a program to read through the **mbox-short.txt** file and figure out who has sent the **greatest number of mail messages**.

* The program looks for **'From'** lines and takes the **second word** of those lines as the person who sent the mail. The program creates a Python dictionary that maps the sender's mail address to a count of the number of times they appear in the file.

```
[4]: # PART1
# Calculate gross pay: regular rate for up to 40 hours, 1.5x for overtime
def compute_pay(arg_hours, arg_rate, base_hours=40, ot_rate=1.5):
    if arg_hours <= base_hours:
        gross_pay = arg_hours * arg_rate
    else:
        overtime_hours = arg_hours - base_hours
        gross_pay=(base_hours*arg_rate) + (overtime_hours*arg_rate*ot_rate)
    return gross_pay
```

```

try:
    # Prompt for input and transform to float
    hours_worked = float(input("Enter hours:"))
    hourly_rate = float(input("Enter rate per hour:"))

    # Print the Input data and Gross pay
    print("-" * 22)
    print(f"Gross pay = {compute_pay(hours_worked, hourly_rate):.2f}")

except ValueError:
    print("Error: Please enter numeric input!")

```

Enter hours: 45

Enter rate per hour: 10.50

Gross pay = 498.75

```

[13]: # PART2
from pathlib import Path

# Constant
FILE_NAME = 'romeo.txt'
file_path = Path(FILE_NAME)
word_list = []

# Check for the existence of the file
if file_path.exists():
    with file_path.open("r", encoding="utf-8") as file_handle:
        for line in file_handle:
            # Remove spaces at the end and split the string into a list of words
            current_line_words = line.split()

            for word in current_line_words:
                # Check if such a word is already in our list
                if word not in word_list:
                    word_list.append(word)

            # Sort the list alphabetically (changes the list itself)
            word_list.sort(key=str.lower)

            # Output the result
            output = ", ".join(word_list)
            print(f"List of words ({len(word_list)}):")
            print(f"[{output}]")

else:
    print(f"Error: File '{FILE_NAME}' not found")

```

List of words (26):

[already, and, Arise, breaks, But, east, envious, fair, grief, is, It, Juliet, kill, light, moon, pale, sick, soft, sun, the, through, what, Who, window, with, yonder]

```
[3]: # PART3
from pathlib import Path

# Constant for searching
FILE_NAME = 'mbox-short.txt'
TARGET_PHRASE = "From "

file_path = Path(FILE_NAME)
counts = {}

# Check for the existence of the file
if file_path.exists():
    # Open the file for reading
    with file_path.open("r", encoding="utf-8") as file_handle:
        for line in file_handle:
            # Search only for lines that begin with "From "
            if line.startswith(TARGET_PHRASE):
                words = line.split()
                # Extract email (second word)
                email = words[1]
                # Update the dictionary: .get(key, 0) will return 0
                # if the key does not yet exist
                counts[email] = counts.get(email, 0) + 1

    # key=counts.get tells the max function to compare not the email names,
    # but their number
    if counts:
        big_sender = max(counts, key=counts.get)
        big_count = max(counts.values())
    else:
        print("No message found")

    # Output the result
    print(f"Most active sender: {big_sender}")
    print(f"Number of messages: {big_count}")

else:
    print(f"Error: File '{FILE_NAME}' not found")
```

Most active sender: cwen@iupui.edu

Number of messages: 5